

Rently — Database Schema Documentation

Living document — update this file as the project evolves.

Last updated: May 2026

Table of Contents

1. [Overview](#)
 2. [User Types](#)
 3. [ERD Diagram](#)
 4. [Tables](#)
 - [users](#)
 - [user_profiles](#)
 - [roles \(Spatie\)](#)
 - [properties](#)
 - [property_images](#)
 - [amenities](#)
 - [amenity_property](#)
 - [property_manager_tenant](#)
 - [leases](#)
 - [documents](#)
 - [work_orders](#)
 - [work_order_updates](#)
 - [conversations](#)
 - [messages](#)
 - [notifications](#)
 - [payments](#)
 5. [Key Design Decisions](#)
 6. [Eloquent Models & Relationships](#)
 7. [Route & Middleware Structure](#)
 8. [Controllers](#)
 9. [Policies](#)
 10. [Notifications](#)
 11. [View Structure](#)
 12. [Feature Progress](#)
-

Overview

Rently is a housing rental portal built in Laravel. It supports three user types — Tenant, Property Manager, and Administrator — each with their own dashboard and permissions.

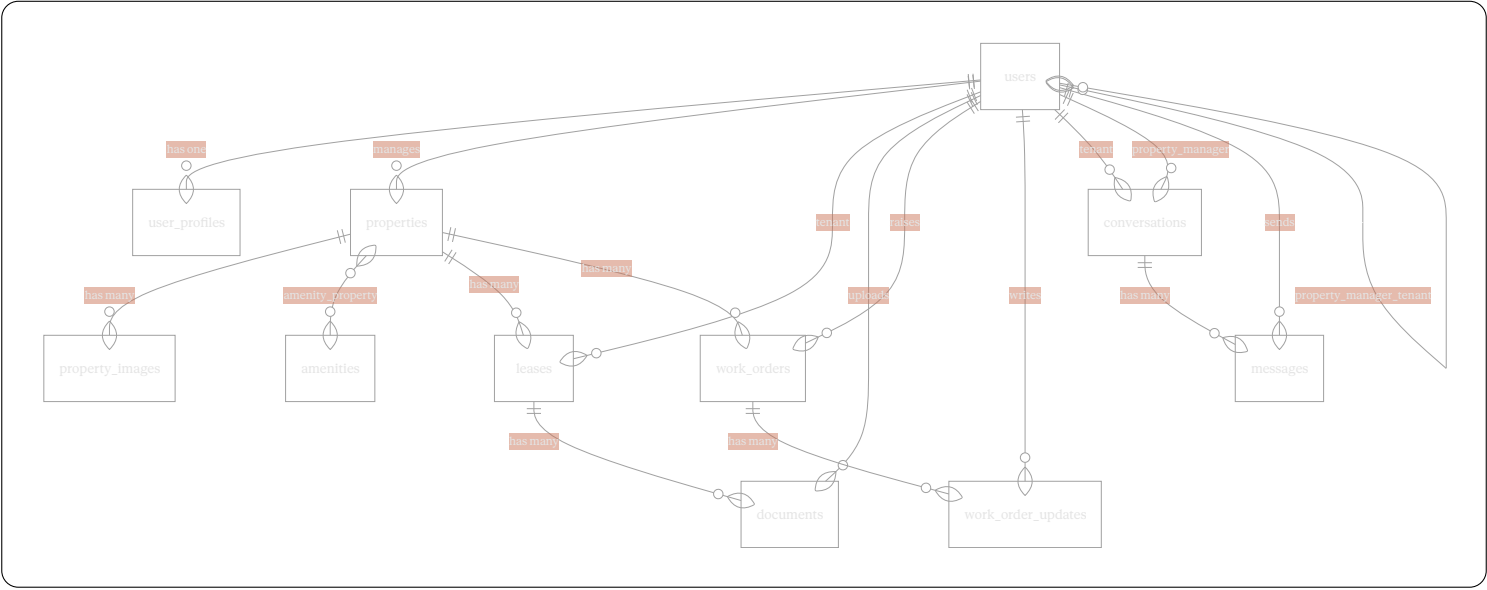
Tech stack

- Laravel 13 + Breeze (auth scaffolding)
- Spatie Laravel Permission (roles)
- MySQL

User Types

Role	Description
admin	Full system access. Manages users, properties, and assignments.
property_manager	Manages their assigned properties and tenants.
tenant	Access to their own lease, documents, messages, and work orders.

ERD Diagram



Tables

users

Scaffolded by Laravel Breeze. Extended with `first_name` and `last_name`.

Column	Type	Notes
<code>id</code>	bigint, PK	
<code>first_name</code>	string	
<code>last_name</code>	string	
<code>email</code>	string, unique	
<code>email_verified_at</code>	timestamp, nullable	
<code>password</code>	string	Hashed
<code>remember_token</code>	string, nullable	
<code>created_at</code>	timestamp	
<code>updated_at</code>	timestamp	

user_profiles

Extended profile data for all user types. Kept separate from `users` to keep the auth table lean.

Column	Type	Notes
<code>id</code>	bigint, PK	
<code>user_id</code>	FK → users	Cascades on delete
<code>legal_name</code>	string	
<code>preferred_name</code>	string, nullable	
<code>phone</code>	string, nullable	
<code>address</code>	string, nullable	Personal home address
<code>emergency_contact_name</code>	string, nullable	
<code>emergency_contact_phone</code>	string, nullable	
<code>emergency_contact_relationship</code>	string, nullable	e.g. spouse, parent, friend
<code>profile_image</code>	text, nullable	Storage path or URL

created_at	timestamp
updated_at	timestamp

roles (Spatie)

Managed by Spatie Laravel Permission. Do not modify manually.

Seeded roles:

Name
admin
property_manager
tenant

Related Spatie tables: permissions, model_has_roles, model_has_permissions, role_has_permissions

properties

A property belongs to a property manager. Can be assigned to a tenant via a lease.

Column	Type	Notes
id	bigint, PK	
property_manager_id	FK → users, nullable	Cascades on delete
title	string	
slug	string, unique	SEO-friendly URL identifier
key_features	text, nullable	Free text / WYSIWYG input
description	longtext	
address	string	
latitude	decimal(10,7), nullable	

longitude	decimal(10,7), nullable	
price	decimal(10,2)	Monthly rent
property_type	enum	house, apartment, studio, commercial
bedrooms	tinyint unsigned	
bathrooms	tinyint unsigned	
size	integer, nullable	Square footage/metres — unit defined at app level
availability_status	enum	available, occupied, under_maintenance
created_at	timestamp	
updated_at	timestamp	

property_images

Stores images for a property. Supports featured image and ordering.

Column	Type	Notes
id	bigint, PK	
property_id	FK → properties	Cascades on delete
path	text	Storage path or URL
is_featured	boolean	Default: false
sort_order	integer	Default: 0
created_at	timestamp	
updated_at	timestamp	

amenities

A seeded lookup table of available amenities.

Column	Type	Notes
<code>id</code>	bigint, PK	
<code>name</code>	string	e.g. Parking, Garden
<code>slug</code>	string, unique	
<code>created_at</code>	timestamp	
<code>updated_at</code>	timestamp	

amenity_property

Pivot table — many-to-many between properties and amenities.

Column	Type	Notes
<code>property_id</code>	FK → properties	Cascades on delete
<code>amenity_id</code>	FK → amenities	Cascades on delete

Composite primary key: `(property_id, amenity_id)`

property_manager_tenant

Pivot table — links a property manager to their assigned tenants. Managed by admin.
Independent of leases so the assignment can exist before a lease is created.

Column	Type	Notes
<code>property_manager_id</code>	FK → users	Cascades on delete
<code>tenant_id</code>	FK → users	Cascades on delete
<code>created_at</code>	timestamp	
<code>updated_at</code>	timestamp	

Composite primary key: `(property_manager_id, tenant_id)`

leases

Links a tenant to a property for a defined period. Core record for tenancy management.

Column	Type	Notes
id	bigint, PK	
property_id	FK → properties	Restrict on delete — preserves lease history
tenant_id	FK → users	
status	enum	pending, active, ended, terminated
rent_amount	decimal(10,2)	
start_date	date	
end_date	date, nullable	
terminated_at	timestamp, nullable	
termination_notes	text, nullable	
created_at	timestamp	
updated_at	timestamp	

documents

Documents uploaded by a property manager for a tenant. Can optionally be tied to a lease. Supports simple signature flow.

Column	Type	Notes
id	bigint, PK	
uploaded_by	FK → users	Cascades on delete
tenant_id	FK → users	Cascades on delete
lease_id	FK → leases, nullable	Null on delete — important docs linked to lease
property_id	FK → properties, nullable	Context anchor when no lease exists
title	string	

<code>title</code>	string	
<code>path</code>	text	Storage path or URL
<code>document_type</code>	string	e.g. <code>tenancy_agreement</code> , <code>epc</code> , <code>welcome_pack</code> . Consider enum once types are finalised
<code>requires_signature</code>	boolean	Default: false
<code>is_signed</code>	boolean	Default: false
<code>signed_at</code>	timestamp, nullable	
<code>created_at</code>	timestamp	
<code>updated_at</code>	timestamp	

Validation note: At least one of `lease_id` or `property_id` should be present. Enforce at app/validation layer.

work_orders

Maintenance/repair requests. Can be raised by either a tenant or property manager.

Column	Type	Notes
<code>id</code>	bigint, PK	
<code>property_id</code>	FK → properties	Restrict on delete — preserve history
<code>lease_id</code>	FK → leases, nullable	Null on delete
<code>raised_by</code>	FK → users	Cascades on delete
<code>assigned_to</code>	FK → users, nullable	Null on delete — property manager or contractor
<code>title</code>	string	
<code>description</code>	text	
<code>priority</code>	enum	<code>low</code> , <code>medium</code> , <code>high</code> , <code>urgent</code>
<code>status</code>	enum	<code>open</code> , <code>in_progress</code> , <code>pending_review</code> , <code>resolved</code> , <code>closed</code>

resolved_at	timestamp, nullable
-------------	------------------------

created_at	timestamp
------------	-----------

updated_at	timestamp
------------	-----------

work_order_updates

Log of comments and updates on a work order. Each entry tracks who posted it.

Column	Type	Notes
id	bigint, PK	
work_order_id	FK → work_orders	Cascades on delete
user_id	FK → users	Cascades on delete
comment	text	
created_at	timestamp	
updated_at	timestamp	

conversations

One conversation thread per tenant/property manager relationship. Not tied to a specific property or lease.

Column	Type	Notes
id	bigint, PK	
tenant_id	FK → users	Cascades on delete
property_manager_id	FK → users	Cascades on delete
last_message_at	timestamp, nullable	Used to sort inboxes by recent activity
created_at	timestamp	
updated_at	timestamp	

messages

Individual messages within a conversation. Supports system-generated messages.

Column	Type	Notes
id	bigint, PK	
conversation_id	FK → conversations	Cascades on delete
sender_id	FK → users	Cascades on delete
body	text	
is_system_message	boolean	Default: false. System messages cannot be replied to
read_at	timestamp, nullable	Null = unread
created_at	timestamp	
updated_at	timestamp	

notifications

Managed by Laravel's built-in notification system. Generated via:

```
bash
php artisan notifications:table
```

Column	Type	Notes
id	uuid, PK	
type	string	Notification class name
notifiable_type	string	Polymorphic — typically App\Models\User
notifiable_id	bigint	
data	text	JSON payload
read_at	timestamp, nullable	
created_at	timestamp	
updated_at	timestamp	

`updated_at`

payments

🕒 **Deferred** — to be designed once core functionality is complete.

Planned links: `lease_id`, `tenant_id`, payment gateway reference, amount, status, due date.

Key Design Decisions

Decision	Reason
Single <code>users</code> table for all roles	Simpler auth flow. Roles managed by Spatie.
<code>property_manager_tenant</code> pivot independent of leases	Admin can assign a tenant to a manager before a lease exists.
<code>restrictOnDelete</code> on <code>property_id</code> in leases	Prevents accidental deletion of a property with active/historical leases.
<code>tenant_id</code> uses <code>constrained('users')</code> explicitly	Laravel cannot infer the <code>users</code> table from a non-standard column name.
<code>last_message_at</code> on conversations	Avoids expensive subqueries when sorting inboxes.
<code>is_system_message</code> on messages	Allows UI to style and disable replies on automated messages (e.g. work order updates).
<code>read_at</code> timestamp instead of boolean	Gives read time for free, not just read state.
<code>document_type</code> as string (not enum yet)	Types not fully defined yet — convert to enum once finalised.
Laravel built-in notifications table	Integrates with <code>Notifiable</code> trait, supports multiple channels out of the box.
Spatie Laravel Permission for roles	Battle-tested, widely supported, clean API (<code>assignRole</code> , <code>hasRole</code>).

Eloquent Models & Relationships

Queries

Overview

Each model maps to a database table and defines its relationships to other models. Laravel infers the table name from the class name automatically (e.g. `WorkOrder` → `work_orders`). Where a foreign key uses a non-standard column name (e.g. `tenant_id`, `raised_by`), the table is passed explicitly to `belongsTo()` to avoid Laravel guessing incorrectly.

User

File: `app/Models/User.php`

Relationship	Type	Notes
<code>properties()</code>	hasMany Property	As property manager
<code>leases()</code>	hasMany Lease	As tenant
<code>profile()</code>	hasOne UserProfile	
<code>tenants()</code>	belongsToMany User	Via <code>property_manager_tenant</code> pivot
<code>propertyManager()</code>	belongsToMany User	Via <code>property_manager_tenant</code> pivot

```
php

// Usage examples
$user->properties;      // all properties managed by this user
$user->leases;           // all leases for this tenant
$user->profile;          // user's profile
$user->tenants;          // tenants assigned to this manager
$user->propertyManager; // manager assigned to this tenant
```

UserProfile

File: `app/Models/UserProfile.php`

Relationship	Type	Notes
<code>user()</code>	belongsTo User	

Property

File: `app/Models/Property.php`

File: `app/Models/Property.php`

Relationship	Type	Notes
<code>propertyManager()</code>	belongsTo User	Via <code>property_manager_id</code>
<code>images()</code>	hasMany PropertyImage	
<code>leases()</code>	hasMany Lease	
<code>workOrders()</code>	hasMany WorkOrder	
<code>amenities()</code>	belongsToMany Amenity	Via <code>amenity_property</code> pivot

```
php

// Usage examples
$property->propertyManager; // the manager who owns this property
$property->images;           // all images for this property
$property->amenities;        // all amenities attached to this property
$property->leases;           // all leases for this property
```

PropertyImage

File: `app/Models/PropertyImage.php`

Relationship	Type	Notes
<code>property()</code>	belongsTo Property	

Amenity

File: `app/Models/Amenity.php`

Relationship	Type	Notes
<code>properties()</code>	belongsToMany Property	Via <code>amenity_property</code> pivot

```
php

// Usage example
$amenity->properties; // all properties with this amenity
```

Lease

File: `app/Models/Lease.php`

Relationship	Type	Notes
<code>property()</code>	belongsTo Property	
<code>tenant()</code>	belongsTo User	Via <code>tenant_id</code>
<code>documents()</code>	hasMany Document	
<code>workOrders()</code>	hasMany WorkOrder	

```
php
// Usage examples
$lease->tenant;    // the tenant on this lease
$lease->property;  // the property this lease is for
$lease->documents; // all documents linked to this lease
```

Document

File: `app/Models/Document.php`

Relationship	Type	Notes
<code>uploadedBy()</code>	belongsTo User	Via <code>uploaded_by</code>
<code>tenant()</code>	belongsTo User	Via <code>tenant_id</code>
<code>lease()</code>	belongsTo Lease	Nullable
<code>property()</code>	belongsTo Property	Nullable

WorkOrder

File: `app/Models/WorkOrder.php`

Relationship	Type	Notes
<code>property()</code>	belongsTo Property	

<code>property()</code>	belongsTo Property	
<code>lease()</code>	belongsTo Lease	Nullable
<code>raisedBy()</code>	belongsTo User	Via <code>raised_by</code>
<code>assignedTo()</code>	belongsTo User	Via <code>assigned_to</code> , nullable
<code>updates()</code>	hasMany WorkOrderUpdate	

php

// Usage examples

`$workOrder->raisedBy;` // user who raised the work order

`$workOrder->assignedTo;` // user assigned to action it

`$workOrder->updates;` // all updates/comments on this work order

WorkOrderUpdate

File: `app/Models/WorkOrderUpdate.php`

Relationship	Type	Notes
<code>workOrder()</code>	belongsTo WorkOrder	
<code>user()</code>	belongsTo User	

Conversation

File: `app/Models/Conversation.php`

Relationship	Type	Notes
<code>tenant()</code>	belongsTo User	Via <code>tenant_id</code>
<code>propertyManager()</code>	belongsTo User	Via <code>property_manager_id</code>
<code>messages()</code>	hasMany Message	
<code>latestMessage()</code>	hasMany Message	Ordered by latest, limit 1. Used for inbox previews.

php

```
// Usage examples
$conversation->messages;      // all messages in this conversation
$conversation->latestMessage; // most recent message for inbox preview
```

Message

File: `app/Models/Message.php`

Relationship	Type	Notes
<code>conversation()</code>	belongsTo Conversation	
<code>sender()</code>	belongsTo User	Via <code>sender_id</code>

Route & Middleware Structure

Authentication

Handled by **Laravel Breeze**. Routes defined in `routes/auth.php`.

After login, users are redirected to their role-specific dashboard via the `AuthenticatedSessionController`.

Role-based Redirects

Role	Redirect
<code>admin</code>	<code>/admin/dashboard</code>
<code>property_manager</code>	<code>/manager/dashboard</code>
<code>tenant</code>	<code>/tenant/dashboard</code>

Route Groups

Routes are grouped by role using Spatie's `role` middleware:

Middleware	Prefix	Route Name Prefix
<code>auth, role:tenant</code>	<code>/tenant</code>	<code>tenant.</code>

auth, role:property_manager

/manager

manager.

auth, role:admin

/admin

admin.

Naming Convention

Routes follow Laravel's named route convention:

```
php

// Examples
route('tenant.dashboard')    // → /tenant/dashboard
route('manager.dashboard')    // → /manager/dashboard
route('admin.dashboard')      // → /admin/dashboard
```

Dashboard Views

Role	View Path
Tenant	resources/views/dashboard/tenant/dashboard.blade.php
Property Manager	resources/views/dashboard/manager/dashboard.blade.php
Admin	resources/views/dashboard/admin/dashboard.blade.php

Controllers

Controllers are organised by role namespace to keep concerns separated.

Public

Controller	Path	Purpose
PropertyController	App\Http\Controllers\PropertyController	Public listings and single property view

Manager

Controller	Path
Manager\PropertyController	App\Http\Controllers\Manager\PropertyController

Manager\LeaseController

App\Http\Controllers\Manager\LeaseController

Manager\DocumentController

App\Http\Controllers\Manager\DocumentController

Manager\WorkOrderController

App\Http\Controllers\Manager\WorkOrderController

Manager\ConversationController

App\Http\Controllers\Manager\ConversationController

Tenant

Controller

Path

Tenant\LeaseController

App\Http\Controllers\Tenant\LeaseController

Tenant\DocumentController

App\Http\Controllers\Tenant\DocumentController

Tenant\WorkOrderController

App\Http\Controllers\Tenant\WorkOrderController

Tenant\ConversationController

App\Http\Controllers\Tenant\ConversationController

Admin

Controller

Path

Pu

Admin\PropertyController

App\Http\Controllers\Admin\PropertyController

Ful
ma

Admin\LeaseController

App\Http\Controllers\Admin\LeaseController

Ful
ma

Admin\DocumentController	App\Http\Controllers\Admin\DocumentController	Doc ove
Admin\WorkOrderController	App\Http\Controllers\Admin\WorkOrderController	Wo ove

Shared

Controller	Path	Pur
MessageController	App\Http\Controllers\MessageController	Ser me: (sh ten ma
WorkOrderUpdateController	App\Http\Controllers\WorkOrderUpdateController	Ad ord upc (sh
UserProfileController	App\Http\Controllers\UserProfileController	Edi (sh all1
NotificationController	App\Http\Controllers\NotificationController	Ma not as r

Policies

Policies control model-level access. Each policy is auto-discovered by Laravel.

Policy	Model	File
PropertyPolicy	Property	app/Policies/PropertyPolicy.php
LeasePolicy	Lease	app/Policies/LeasePolicy.php
DocumentPolicy	Document	app/Policies/DocumentPolicy.php
WorkOrderPolicy	WorkOrder	app/Policies/WorkOrderPolicy.php
MessagePolicy	Message	app/Policies/MessagePolicy.php

Policy rules summary

PropertyPolicy

- `viewAny` / `view` — public, including guests (`?User`)
- `create` / `update` — admin or property manager (managers limited to own properties)
- `delete` — admin only

LeasePolicy

- `view` — admin sees all, manager sees own properties, tenant sees own leases
- `create` / `update` — admin or property manager
- `delete` — admin only

DocumentPolicy

- `view` — admin, uploader (manager), or assigned tenant
- `create` / `update` / `delete` — admin or uploading manager

WorkOrderPolicy

- `view` — admin sees all, manager sees own properties, tenant sees own raised orders
- `create` — manager or tenant
- `update` — admin or manager (own properties)
- `delete` — admin only

MessagePolicy

- `view` — admin, or either party in the conversation
- `create` — manager or tenant
- `update` — always false (messages cannot be edited)
- `delete` — admin only

Notifications

Notifications use Laravel's built-in database channel. All notification classes live in `app/Notifications/`.

Notification	Trigger	Recipient
<code>NewMessageNotification</code>	Message sent	Other party in conversation
<code>WorkOrderCreatedNotification</code>	Work order raised	Property manager

WorkOrderUpdatedNotification	Work order status changed	Tenant who raised it
WorkOrderCommentNotification	Update/comment added	Other party on work order
DocumentUploadedNotification	Document uploaded	Tenant
DocumentSignedNotification	Document signed	Uploading manager
LeaseStatusChangedNotification	Lease status updated	Tenant

Notification data payload

Each notification stores a `type` key in its data payload used by `NotificationController` to redirect to the correct resource on click:

Type	Redirect
<code>new_message</code>	conversation show page
<code>work_order_created</code> / <code>work_order_updated</code> / <code>work_order_comment</code>	work order show page
<code>document_uploaded</code> / <code>document_signed</code>	document show page
<code>lease_status_changed</code>	lease show page

Route prefix helper

The `User` model has a `routePrefix()` helper method to map role names to route prefixes:

```
php
$user->routePrefix(); // 'manager', 'tenant', or 'admin'
```

This avoids issues with `property_manager` role name not matching the `manager` route prefix.

View Structure

```
resources/views/
├── dashboard/
│   ├── admin/
│   │   └── dashboard.blade.php
```



Feature Progress

Feature	Status	Branch
Database schema & migrations	✔ Complete	database
Seed data	✔ Complete	database
Eloquent models & relationships	✔ Complete	feature/eloquent-models
Spatie roles	✔ Complete	feature/eloquent-models
Route protection & middleware	✔ Complete	feature/auth-middleware
Policies	✔ Complete	feature/auth-middleware
Property CRUD (public, manager, admin)	✔ Complete	feature/property-crud

Dashboard structure (admin, manager, tenant)	 Complete	feature/property-crud
Messaging system	 Complete	feature/messaging
Lease management	 Complete	feature/lease-management
Work orders	 Complete	feature/work-orders
Documents with signing	 Complete	feature/documents
Tenant dashboard	 Complete	feature/tenant-dashboard
User profile management	 Complete	feature/user-profiles
Navigation	 Complete	feature/navigation
Notifications	 Complete	feature/notifications
Admin user management	 Pending	—
Payments	 Deferred	—